

LeetCode 4058

Maximum Pulse Value After One Subarray Rotation

Medium · Weekly Contest 520 (Q3) · Prefix sums + index-parity trick · C++

1. Problem

The **pulse value** of an array is the alternating sum starting with plus: $a[0] - a[1] + a[2] - a[3] + \dots$

You may do **at most one** operation on **nums**: pick indices $l < r$ and **left-rotate** the subarray $\text{nums}[l..r]$ by exactly one position, i.e. $[a,b,c,d]$ becomes $[b,c,d,a]$ (the first element goes to the back). Return the maximum pulse value you can get. **Constraints:** $1 \leq n \leq 10^5$, $-10^9 \leq \text{nums}[i] \leq 10^9$.

Input	Output	Why
[1,5,2]	6	rotate $\text{nums}[0..1] \rightarrow [5,1,2]$, pulse = $5 - 1 + 2 = 6$ (original is -2)
[6,4,3]	7	rotate $\text{nums}[1..2] \rightarrow [6,3,4]$, pulse = $6 - 3 + 4 = 7$ (original is 5)
[9,7]	2	original $9 - 7 = 2$ is already best, so do nothing

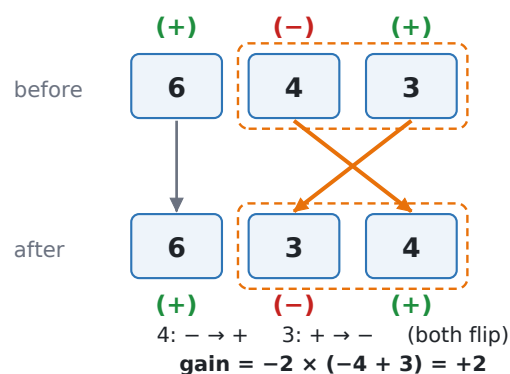
2. Intuition

Obvious approach. Try every (l, r) , rotate, recompute the pulse: $O(n^3)$. Even if each pair is evaluated in $O(1)$ it is still $O(n^2)$, too slow for $n = 10^5$.

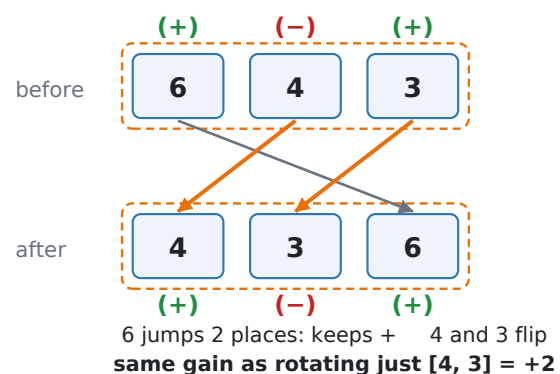
Look at what a rotation really changes. The pulse is $\sum \text{sign}(k) \cdot \text{nums}[k]$ where $\text{sign}(k) = +$ for even k and $-$ for odd k . An element's contribution changes only if its **index parity** changes, i.e. it moves an **odd** distance. In a left-rotation of $[l..r]$:

- elements $l+1 \dots r$ each move **one step left** $\rightarrow$ parity flips $\rightarrow$ their sign flips (gain $-2 \cdot \text{sign} \cdot \text{value}$ each);
- the element at l jumps to r , a distance of $r-l \rightarrow$ its sign flips only if $r-l$ is **odd**, i.e. the segment length $r-l+1$ is **even**.

Even length: rotate $\text{nums}[1..2] = [4, 3]$



Odd length: rotate $\text{nums}[0..2] = [6, 4, 3]$



$(+)(-)$ = sign of that position. Orange arrows = elements whose sign flips. Both rotations gain $+2$.

Even length $\rightarrow$ every element in the segment flips sign $\rightarrow$ **gain = $-2 \times$ (signed sum of the segment)**.

Odd length $\rightarrow$ the first element keeps its sign, the rest flip $\rightarrow$ the gain equals that of the even-length segment $[l+1..r]$. So odd lengths never give anything new.

Turn it into prefix sums. Let $P[i]$ = signed sum of the first i elements ($P[0] = 0$, and element k is added with $+$ if k is even, $-$ if odd). The even-length segment $\text{nums}[j..i-1]$ has signed sum $P[i] - P[j]$, so

gain(j, i) = $-2 \cdot (P[i] - P[j]) = 2 \cdot (P[j] - P[i])$, for $j < i$ with $i - j$ even (i and j have the **same parity**).

So we need the best pair $j < i$ of the same parity that maximises $P[j] - P[i]$: a classic “best earlier value” scan, done separately for even and odd j . Because the operation is “at most one”, a gain of 0 (do nothing) is always allowed.

3. Approach (what the code does)

1. `base` = pulse of the original array (lines 7–11).
2. Scan $i = 1 \dots n$ and build `pref = P[i]`: for odd i add `nums[i-1]`, for even i subtract it (lines 16–17).
3. Keep two buckets by parity: `maxPref[p] = largest P[j]` seen so far with $j \% 2 == p$. `maxPref[0]` starts at 0 (that is `P[0]`); `maxPref[1]` starts as `LLONG_MIN` meaning “empty”.
4. If the bucket of i 's parity is not empty: `gain = -2*(pref - maxPref[parity])`, `bestGain = max(bestGain, gain)` (lines 19–20). `bestGain` starts at 0 = “do nothing”.
5. **Then** update `maxPref[parity]` with `pref` (line 22). Updating after using it guarantees $j < i$. Finally return `base + bestGain`.

4. State / Formula

Name	Meaning
<code>pref</code>	$P[i]$, the signed prefix sum after i elements (long long)
<code>maxPref[0], maxPref[1]</code>	best $P[j]$ so far for even j / odd j ($j < i$)
<code>bestGain</code>	best improvement found so far; starts at 0 = no operation

- **Transition** (for each i): `candidate = 2*(maxPref[i%2] - P[i])`; `bestGain = max(bestGain, candidate)`; then `maxPref[i%2] = max(maxPref[i%2], P[i])`. A best pair (j, i) means: rotate `nums[j..i-1]` (even length).
- **Base cases**: `P[0] = 0` sits in the even bucket, the odd bucket is empty, `bestGain = 0`. **Order**: one left-to-right pass, $i = 1 \dots n$. No recursion.

5. Code

Exactly as in your ZIP (LC_4058/code.cpp). The line numbers on the left are only for reference.

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  class Solution {
4  public:
5      long long maxValue(vector<int>& nums) {
6          int n = nums.size();
7          long long base = 0;
8          for(int i=0;i<n;i++){
9              if(i%2==0) base+=nums[i];
10             else base-=nums[i];
11         }
12         long long bestGain = 0;
13         long long pref = 0;
14         long long maxPref[2] = {0,LLONG_MIN};
15         for(int i=1;i<=n;i++){
16             if(i%2==1) pref+=nums[i-1];
17             else pref-=nums[i-1];
18             int parity = i%2;
19             if(maxPref[parity]!=LLONG_MIN)
20                 bestGain=max(bestGain,-2LL*(pref-maxPref[parity]));
21
22             maxPref[parity]=max(maxPref[parity],pref);
23         }
24         return base+bestGain;
25     }
26 };

```

6. Dry Run

nums = [6, 4, 3], expected 7. base = 6 − 4 + 3 = 5. Prefix values: P[0]=0, P[1]=6, P[2]=2, P[3]=5.

i	nums[i−1]	pref = P[i]	parity	maxPref before [even, odd]	candidate gain	bestGain	maxPref after
1	6	6	1	[0, empty]	bucket empty → skip	0	[0, 6]
2	4	2	0	[0, 6]	$-2 \cdot (2 - 0) = -4$	0	[2, 6]
3	3	5	1	[2, 6]	$-2 \cdot (5 - 6) = +2$	2	[2, 6]

Answer = 5 + 2 = **7** ✓. The winning pair is j = 1 (P = 6) and i = 3 (P = 5) → rotate nums[1..2] = [4, 3] into [3, 4] → [6, 3, 4] → 6 − 3 + 4 = 7.

Two quick checks with the same recipe:

- [1,5,2]: base = −2, P = 0, 1, −4, −2. Best pair j=0, i=2 → gain $2 \cdot (0 - (-4)) = 8$ → −2 + 8 = **6** ✓ (rotate nums[0..1] → [5,1,2]).
- [9,7]: base = 2, P = 0, 9, 2. Only pair j=0, i=2 → gain $2 \cdot (0 - 2) = -4 < 0$, so bestGain stays 0 → answer **2** ✓ (do nothing).

7. Complexity

	Cost	Reason
Time	O(n)	one pass for base, one pass for the scan
Space	O(1)	a few long long variables and a 2-element array
Recursion stack	none	plain loops

- Overflow: |P[i]| can reach $10^5 \cdot 10^9 = 10^{14}$, and a gain can reach a few 10^{14} . Use `long long`; `-2LL*` forces 64-bit math. The `!= LLONG_MIN` check on line 19 keeps the empty bucket from ever being subtracted (that would overflow).
- Edge cases: n = 1 has no valid (l, r), the odd bucket is empty at i = 1, so the answer is nums[0]. If every rotation makes things worse, bestGain stays 0.

8. Quick Revision

Core idea:	Only index-parity changes matter. Rotating an even-length segment flips the sign of every element in it (odd length adds nothing new), so gain = $-2 \times$ (signed sum of an even-length segment).
State:	pref = P[i] (signed prefix sum); maxPref[0/1] = best earlier P[j] with j even / odd; bestGain (starts at 0).
Transition:	gain = $-2 \cdot (\text{pref} - \text{maxPref}[i\%2])$; bestGain = max(bestGain, gain); then update maxPref[i%2].
Important observation:	same parity of j and i ⇔ even-length segment. “At most one” means gain 0 is allowed.
TC:	O(n)
SC:	O(1)